

# CS306 MCQs - Part 2 (Topics 4-20, MCQs 51-100)

## Question No: 1 (Marks: 1)

Which term best completes the following statement: 'Class vs Instance Attributes Instance Attributes: Defined in the `__init__()` method and are unique to each object. Class Attributes: Defined outside any methods and shared across all instances. • • Example of Class Attribute: `class Dog: species = "Canine"` # class attribute `def __init__(self, name): self.name = name` # instance attribute Access: `d = Dog("Buddy") print(d.name) print(d.species)` # Output: Buddy # Output: Canine ■ 7.?'

- A. first
- B. structures
- C. solution

**D. Buddy**

### Explanation:

The correct answer, 'Buddy', is directly supported by the provided text: 'Class vs Instance Attributes Instance Attributes: Defined in the `__init__()` method and are unique to each object. Class Attributes: Defined outside any methods and shared across all instances. • • Example of Class Attribute: `class Dog: species = "Canine"` # class attribute `def __init__(self, name): self.name = name` # instance attribute Access: `d = Dog("Buddy") print(d.name) print(d.species)` # Output: Buddy # Output: Canine ■ 7.?'

## Question No: 2 (Marks: 1)

Which term best completes the following statement: 'Part 1 ■ Lecture Objectives • • • • Understand the basic concepts of Object-\_\_\_\_\_ Programming (OOP). Learn how to define and create classes and objects in Python. Understand attributes and methods. Learn the concept of self and constructors (`__init__` method). ■ 1.?'

**A. Oriented**

- B. lecture
- C. number
- D. initializes

### Explanation:

The correct answer, 'Oriented', is directly supported by the provided text: 'Part 1 ■ Lecture Objectives • • • • Understand the basic concepts of Object-Oriented Programming (OOP). Learn how to define and create classes and objects in Python. Understand attributes and methods. Learn the concept of self and constructors (`__init__` method). ■ 1.?'

## Question No: 3 (Marks: 1)

Which term best completes the following statement: 'It must be the first parameter of any method in the \_\_\_\_\_. Example: class Student: def \_\_init\_\_(self, name): self.name = name def greet(self): print(f"Hello, my name is {self.name}") Usage: s = Student("Ayesha") s.greet() # Output: Hello, my name is Ayesha Notes for CS306 introduction to python Noor institute of technology ■ 5.'

**A. Class**

- B. initializes
- C. def
- D. each

**Explanation:**

The correct answer, 'Class', is directly supported by the provided text: 'It must be the first parameter of any method in the class. Example: class Student: def \_\_init\_\_(self, name): self.name = name def greet(self): print(f"Hello, my name is {self.name}") Usage: s = Student("Ayesha") s.greet() # Output: Hello, my name is Ayesha Notes for CS306 introduction to python Noor institute of technology ■ 5.'

#### Question No: 4 (Marks: 1)

Which term best completes the following statement: 'Example: Simple \_\_\_\_\_ class Animal: def speak(self): print("Animal speaks") class Dog(Animal): pass d = Dog() d.speak() # Output: Animal speaks Here, Dog inherits the speak() method from Animal.'

A. self

**B. Inheritance**

- C. create
- D. inherit

**Explanation:**

The correct answer, 'Inheritance', is directly supported by the provided text: 'Example: Simple Inheritance class Animal: def speak(self): print("Animal speaks") class Dog(Animal): pass d = Dog() d.speak() # Output: Animal speaks Here, Dog inherits the speak() method from Animal.'

#### Question No: 5 (Marks: 1)

Which term best completes the following statement: 'Inheritance in Python Inheritance allows a class (child/subclass) to inherit attributes and methods from another class (parent/superclass). Benefits of Inheritance: ••• Code reusability Better organization of \_\_\_\_\_ Supports extensibility Syntax: class Parent: # parent class code class Child(Parent): Notes for CS306 introduction to python Noor institute of technology # child class code ■ 2.'

**A. Code**

- B. better
- C. lecture
- D. explore

**Explanation:**

The correct answer, 'Code', is directly supported by the provided text: 'Inheritance in Python Inheritance allows a class (child/subclass) to inherit attributes and methods from another class (parent/superclass).

Benefits of Inheritance: • • • Code reusability Better organization of code Supports extensibility Syntax:  
class Parent: # parent class code  
class Child(Parent): # child class code

### Question No: 6 (Marks: 1)

Which term best completes the following statement: '(Inheritance and Polymorphism) ■ Lecture Objectives • • • • Understand the concept of inheritance in Python. Learn how to create subclasses. Explore the use of the super() \_\_\_\_\_. Understand polymorphism and method overriding. ■ 1.'?

- A. inheritance
- B. parent

**C. Function**

- D. allows

#### Explanation:

The correct answer, 'Function', is directly supported by the provided text: '(Inheritance and Polymorphism) ■ Lecture Objectives • • • • Understand the concept of inheritance in Python. Learn how to create subclasses. Explore the use of the super() function. Understand polymorphism and method overriding. ■ 1.'.

### Question No: 7 (Marks: 1)

Which term best completes the following statement: 'Access \_\_\_\_\_ in Python Python uses conventions (not strict rules) to denote access control: Modifier Syntax Public var Access Level Accessible anywhere Protected \_var Accessible in subclass Private \_\_var Name mangled, internal use only ■ 4.'?

**A. Modifiers**

- B. oriented
- C. direct
- D. mangling

#### Explanation:

The correct answer, 'Modifiers', is directly supported by the provided text: 'Access Modifiers in Python Python uses conventions (not strict rules) to denote access control: Modifier Syntax Public var Access Level Accessible anywhere Protected \_var Accessible in subclass Private \_\_var Name mangled, internal use only ■ 4.'.

### Question No: 8 (Marks: 1)

Which term best completes the following statement: 'Summary • • • • \_\_\_\_\_ keeps data safe from unintended access or modification. Python supports public, protected, and private attributes (by convention). Use getter/setter methods or @property decorators to access or update private variables safely. Data hiding improves program security and flexibility. Notes for CS306 introduction to python Noor institute of technology ■ Homework / Practice'?

- A. code

- B. use
- C. lecture

#### ***D. Encapsulation***

#### **Explanation:**

The correct answer, 'Encapsulation', is directly supported by the provided text: 'Summary •••• Encapsulation keeps data safe from unintended access or modification. Python supports public, protected, and private attributes (by convention). Use getter/setter methods or @property decorators to access or update private variables safely. Data hiding improves program security and flexibility. Notes for CS306 introduction to python Noor institute of technology ■ Homework / Practice'.

#### **Question No: 9 (Marks: 1)**

Which term best completes the following statement: 'Properties (Alternative to Getters/Setters) Python provides a built-in @property decorator. Notes for CS306 introduction to python Noor institute of technology \_\_\_\_\_ BankAccount: def \_\_init\_\_(self): self.\_\_balance = 0 @property def balance(self): return self.\_\_balance @balance.setter def balance(self, amount): if amount >= 0: self.\_\_balance = amount else: print("Invalid amount") acc = BankAccount() acc.balance = 500 print(acc.balance) # Output: 500 ■ 9.'?

- A. \_\_name
- B. decorator
- C. methods

#### ***D. Class***

#### **Explanation:**

The correct answer, 'Class', is directly supported by the provided text: 'Properties (Alternative to Getters/Setters) Python provides a built-in @property decorator. Notes for CS306 introduction to python Noor institute of technology class BankAccount: def \_\_init\_\_(self): self.\_\_balance = 0 @property def balance(self): return self.\_\_balance @balance.setter def balance(self, amount): if amount >= 0: self.\_\_balance = amount else: print("Invalid amount") acc = BankAccount() acc.balance = 500 print(acc.balance) # Output: 500 ■ 9.'.

#### **Question No: 10 (Marks: 1)**

Which term best completes the following statement: 'Importance of \_\_\_\_\_ ••• Controls access to the internal state of objects. Improves code maintainability and flexibility. Protects object integrity by preventing unintended changes. ■■■ 3.'?

- A. built
- B. with
- C. variables

#### ***D. Encapsulation***

#### **Explanation:**

The correct answer, 'Encapsulation', is directly supported by the provided text: 'Importance of Encapsulation ••• Controls access to the internal state of objects. Improves code maintainability and flexibility. Protects object integrity by preventing unintended changes. ■■■ 3.'.

**Question No: 11 (Marks: 1)**

Which term best completes the following statement: 'Getter and Setter Methods Used to safely access or modify private attributes. \_\_\_\_\_: class BankAccount: def \_\_init\_\_(self): self.\_\_balance = 0 def get\_balance(self): return self.\_\_balance def set\_balance(self, amount): if amount >= 0: self.\_\_balance = amount else: print("Invalid amount") acc = BankAccount() acc.set\_balance(1000) print(acc.get\_balance()) # Output: 1000 ■ 8.'?

A. sara

**B. Example**

C. helps

D. display

**Explanation:**

The correct answer, 'Example', is directly supported by the provided text: 'Getter and Setter Methods Used to safely access or modify private attributes. Example: class BankAccount: def \_\_init\_\_(self): self.\_\_balance = 0 def get\_balance(self): return self.\_\_balance def set\_balance(self, amount): if amount >= 0: self.\_\_balance = amount else: print("Invalid amount") acc = BankAccount() acc.set\_balance(1000) print(acc.get\_balance()) # Output: 1000 ■ 8.'

**Question No: 12 (Marks: 1)**

Which term best completes the following statement: '■ Lecture Objectives • • • • Understand the concept of encapsulation in object-oriented programming. Learn how to implement data hiding in Python. \_\_\_\_\_ between public, protected, and private attributes. Use getter and setter methods to control access. ■ 1.'?

**A. Differentiate**

B. methods

C. decorator

D. program

**Explanation:**

The correct answer, 'Differentiate', is directly supported by the provided text: '■ Lecture Objectives • • • • Understand the concept of encapsulation in object-oriented programming. Learn how to implement data hiding in Python. Differentiate between public, protected, and private attributes. Use getter and setter methods to control access. ■ 1.'

**Question No: 13 (Marks: 1)**

Which term best completes the following statement: 'Python does name mangling, making the variable harder to access from outside. class Person: def \_\_init\_\_(self, name): self.\_\_name = name # private p = Person("\_\_\_\_\_") # print(p.\_\_name) # Error: AttributeError # Access using name mangling print(p.\_Person\_\_name) # Output: Usman ■■ 7.'?

A. python

B. importance

C. output

**D. Usman**

**Explanation:**

The correct answer, 'Usman', is directly supported by the provided text: 'Python does name mangling, making the variable harder to access from outside. class Person: def \_\_init\_\_(self, name): self.\_\_name = name # private p = Person("Usman") # print(p.\_\_name) # Error: AttributeError # Access using name mangling print(p.\_Person\_\_name) # Output: Usman ■ 7.'

### Question No: 14 (Marks: 1)

Which term best completes the following statement: 'Public \_\_\_\_\_ Accessible from anywhere. class Person: def \_\_init\_\_(self, name): self.name = name # public p = Person("Ali") print(p.name) # Output: Ali ■ 5.'

A. syntax

B. single

**C. Members**

D. \_person\_\_name

**Explanation:**

The correct answer, 'Members', is directly supported by the provided text: 'Public Members Accessible from anywhere. class Person: def \_\_init\_\_(self, name): self.name = name # public p = Person("Ali") print(p.name) # Output: Ali ■ 5.'

### Question No: 15 (Marks: 1)

Which term best completes the following statement: 'Writing to a File Example: file = open("data.txt", "w") file.write("Hello, world!") file.close() \_\_\_\_\_ or overwrites data.txt with the content. ■ 4.'

**A. Creates**

B. like

C. creating

D. modes

**Explanation:**

The correct answer, 'Creates', is directly supported by the provided text: 'Writing to a File Example: file = open("data.txt", "w") file.write("Hello, world!") file.close() Creates or overwrites data.txt with the content. ■ 4.'

### Question No: 16 (Marks: 1)

Which term best completes the following statement: '\_\_\_\_\_ to a File file = open("data.txt", "a") file.write("\nNew line added.") file.close() ■ 6.'

A. about

B. homework

### **C. Appending**

D. modes

#### **Explanation:**

The correct answer, 'Appending', is directly supported by the provided text: 'Appending to a File file = open("data.txt", "a") file.write("\nNew line added.") file.close()' ■ 6.'

### **Question No: 17 (Marks: 1)**

Which term best completes the following statement: 'Introduction to File Handling File handling is used to store data permanently into files (external storage like .txt, .csv, etc.). Python provides built-in functions \_\_\_\_\_: ..... Creating files Writing to files Reading from files Appending data Closing files ■ 2.'?

- A. lists
- B. hello

### **C. For**

D. example

#### **Explanation:**

The correct answer, 'For', is directly supported by the provided text: 'Introduction to File Handling File handling is used to store data permanently into files (external storage like .txt, .csv, etc.). Python provides built-in functions for: ..... Creating files Writing to files Reading from files Appending data Closing files ■ 2.'.

### **Question No: 18 (Marks: 1)**

Which term best completes the following statement: 'Example: Write & Read Student Records \_\_\_\_\_: with open("students.txt", "w") as file: file.write("Ali, 85\n") file.write("Sara, 92\n") Reading: with open("students.txt", "r") as file: for line in file: print(line.strip()) ■ 9.'?

- A. provides
- B. binary
- C. example

### **D. Writing**

#### **Explanation:**

The correct answer, 'Writing', is directly supported by the provided text: 'Example: Write & Read Student Records Writing: with open("students.txt", "w") as file: file.write("Ali, 85\n") file.write("Sara, 92\n") Reading: with open("students.txt", "r") as file: for line in file: print(line.strip()) ■ 9.'.

### **Question No: 19 (Marks: 1)**

Which term best completes the following statement: 'Using with \_\_\_\_\_ (Best Practice) Automatically closes the file after execution. Notes for CS306 introduction to python Noor institute of technology with open("data.txt", "r") as file: print(file.read()) For Writing: with open("data.txt", "w") as file:

file.write("Overwritten content.") ■■ 7.?

- A. used
- B. found
- C. does

**D. Statement**

**Explanation:**

The correct answer, 'Statement', is directly supported by the provided text: 'Using with Statement (Best Practice) Automatically closes the file after execution. Notes for CS306 introduction to python Noor institute of technology with open("data.txt", "r") as file: print(file.read()) For Writing: with open("data.txt", "w") as file: file.write("Overwritten content.") ■■ 7.'

**Question No: 20 (Marks: 1)**

Which term best completes the following statement: 'Reading from a File \_\_\_\_\_ = open("data.txt", "r") content = file.read() print(content) file.close()' Other Read Methods: ••• read(): Reads entire file readline(): Reads one line readlines(): Reads all lines as a list ■ 5.?

- A. added

**B. File**

- C. exist
- D. hello

**Explanation:**

The correct answer, 'File', is directly supported by the provided text: 'Reading from a File file = open("data.txt", "r") content = file.read() print(content) file.close()' Other Read Methods: ••• read(): Reads entire file readline(): Reads one line readlines(): Reads all lines as a list ■ 5.'

**Question No: 21 (Marks: 1)**

Which term best completes the following statement: 'Summary •••• Python provides open() for file handling with multiple modes (r, w, a, etc.). Always close the file or \_\_\_\_\_ with to do it automatically. Use exception handling to manage file errors. Files can be read as full text, line-by-line, or into lists. Binary files use 'rb' and 'wb' modes. Notes for CS306 introduction to python Noor institute of technology ■ Homework / Practice'?

- A. with
- B. from

**C. Use**

- D. statement

**Explanation:**

The correct answer, 'Use', is directly supported by the provided text: 'Summary •••• Python provides open() for file handling with multiple modes (r, w, a, etc.). Always close the file or use with to do it automatically. Use exception handling to manage file errors. Files can be read as full text, line-by-line, or into lists. Binary files use 'rb' and 'wb' modes. Notes for CS306 introduction to python Noor institute of technology ■ Homework / Practice'.



**Question No: 22 (Marks: 1)**

Which term best completes the following statement: 'Working with Binary \_\_\_\_\_ with open("image.jpg", "rb") as f: data = f.read() with open("copy.jpg", "wb") as f: f.write(data) ■ 10.'

**A. Files**

- B. readline
- C. practice
- D. txt

**Explanation:**

The correct answer, 'Files', is directly supported by the provided text: 'Working with Binary Files with open("image.jpg", "rb") as f: data = f.read() with open("copy.jpg", "wb") as f: f.write(data) ■ 10.'

**Question No: 23 (Marks: 1)**

Which term best completes the following statement: 'Catching \_\_\_\_\_ Exceptions try: x = int("abc") except ValueError: print("Invalid input.") except ZeroDivisionError: print("Division error.") You can also combine them: try: # some code except (ValueError, ZeroDivisionError) as e: print("Error:", e) ■ 4.'

- A. crashes
- B. complete

**C. Multiple**

- D. what

**Explanation:**

The correct answer, 'Multiple', is directly supported by the provided text: 'Catching Multiple Exceptions try: x = int("abc") except ValueError: print("Invalid input.") except ZeroDivisionError: print("Division error.") You can also combine them: try: # some code except (ValueError, ZeroDivisionError) as e: print("Error:", e) ■ 4.'

**Question No: 24 (Marks: 1)**

Which term best completes the following statement: 'Summary ••••• Use try...except to catch and handle exceptions. Use else for code that runs if no exception occurs. Use finally to execute cleanup code regardless of exceptions. Raise exceptions manually using \_\_\_\_\_. Knowing built-in exceptions helps debug effectively. ■ Homework / Practice'?

**A. Raise**

- B. understand
- C. occur
- D. may

**Explanation:**

The correct answer, 'Raise', is directly supported by the provided text: 'Summary ••••• Use try...except to catch and handle exceptions. Use else for code that runs if no exception occurs. Use

finally to execute cleanup code regardless of exceptions. Raise exceptions manually using raise. Knowing built-in exceptions helps debug effectively. ■ Homework / Practice'.

### Question No: 25 (Marks: 1)

Which term best completes the following statement: '■ Lecture Objectives •••• Understand what \_\_\_\_\_s are and why they occur. Learn how to handle exceptions using try, except, finally, and else. Explore built-in exception types. Raise custom exceptions using raise. ■■ 1.'?

A. crashes

**B. Exception**

C. combine

D. num

#### Explanation:

The correct answer, 'Exception', is directly supported by the provided text: '■ Lecture Objectives •••• Understand what exceptions are and why they occur. Learn how to handle exceptions using try, except, finally, and else. Explore built-in exception types. Raise custom exceptions using raise. ■■ 1.'.

### Question No: 26 (Marks: 1)

Which term best completes the following statement: 'Example: File Handling with \_\_\_\_\_ try: with open("marks.txt", "r") as file: data = file.read() print(data) except FileNotFoundError: print("The file was not found.") finally: print("File operation complete.") ■ 9.'?

A. raise

**B. Exception**

C. what

D. built

#### Explanation:

The correct answer, 'Exception', is directly supported by the provided text: 'Example: File Handling with Exception try: with open("marks.txt", "r") as file: data = file.read() print(data) except FileNotFoundError: print("The file was not found.") finally: print("File operation complete.") ■ 9.'.

### Question No: 27 (Marks: 1)

Which term best completes the following statement: 'What is an Exception? An exception is an error that occurs during the execution of a program, disrupting its normal flow. \_\_\_\_\_: x = 10 / 0 # ZeroDivisionError Without handling, the program crashes. ■ 2.'?

**A. Example**

B. that

C. number

D. zero

#### Explanation:

The correct answer, 'Example', is directly supported by the provided text: 'What is an Exception? An exception is an error that occurs during the execution of a program, disrupting its normal flow. Example: `x = 10 / 0` # ZeroDivisionError Without handling, the program crashes. ■ 2.'

### Question No: 28 (Marks: 1)

Which term best completes the following statement: 'Using \_\_\_\_\_ Clause Executes if no exception occurs. try: num = int(input("Enter a number: ")) except ValueError: print("Invalid number.") else: print("Square is", num \*\* 2) ■ 5.?'

- A. keyerror
- B. syntax

**C. Else**

- D. its

#### Explanation:

The correct answer, 'Else', is directly supported by the provided text: 'Using else Clause Executes if no exception occurs. try: num = int(input("Enter a number: ")) except ValueError: print("Invalid number.") else: print("Square is", num \*\* 2) ■ 5.'

### Question No: 29 (Marks: 1)

Which term best completes the following statement: 'Exception Handling Syntax try: # Code that may raise an exception except ExceptionType: # Code to handle the exception Example: try: a = int(input("Enter a number: ")) b = 10 / a print("Result:", b) except ZeroDivisionError: print("You cannot divide by zero.") Notes for CS306 introduction to \_\_\_\_\_ Noor institute of technology ■ 3.?'

- A. lecture

**B. Python**

- C. finally
- D. you

#### Explanation:

The correct answer, 'Python', is directly supported by the provided text: 'Exception Handling Syntax try: # Code that may raise an exception except ExceptionType: # Code to handle the exception Example: try: a = int(input("Enter a number: ")) b = 10 / a print("Result:", b) except ZeroDivisionError: print("You cannot divide by zero.") Notes for CS306 introduction to python Noor institute of technology ■ 3.'

### Question No: 30 (Marks: 1)

Which term best completes the following statement: 'Using finally Clause Always executes, regardless of whether an exception occurred. try: file = open("sample.txt", "r") content = file.read() except FileNotFoundError: \_\_\_\_\_("File not found.") finally: print("Cleaning up.") ■■ 6.?'

- A. effectively

**B. Print**

- C. they

D. what

**Explanation:**

The correct answer, 'Print', is directly supported by the provided text: 'Using finally Clause Always executes, regardless of whether an exception occurred. try: file = open("sample.txt", "r") content = file.read() except FileNotFoundError: print("File not found.") finally: print("Cleaning up.") ■ 6.'

**Question No: 31 (Marks: 1)**

Which term best completes the following statement: 'The random \_\_\_\_\_ Used for generating random numbers. Example: import random print(random.randint(1, 10)) # Random int between 1 and 10 Other Functions: Function random.random() Description Float between 0.0 to 1.0 random.choice(list) Randomly select from list random.shuffle(list) Shuffle list ■ 6.'

**A. Module**

B. what

C. called

D. named

**Explanation:**

The correct answer, 'Module', is directly supported by the provided text: 'The random Module Used for generating random numbers. Example: import random print(random.randint(1, 10)) # Random int between 1 and 10 Other Functions: Function random.random() Description Float between 0.0 to 1.0 random.choice(list) Randomly select from list random.shuffle(list) Shuffle list ■ 6.'

**Question No: 32 (Marks: 1)**

Which term best completes the following statement: '\_\_\_\_\_\_ == "\_\_main\_\_" Used to separate \_\_\_\_\_ usage and script execution. # file: greet.py def hello(): print("Hello!") if \_\_\_\_ == "\_\_main\_\_": hello() When greet.py is imported in another file, hello() won't run unless called directly. ■ 9.'

A. import

B. functions

C. etc

**D. Module**

**Explanation:**

The correct answer, 'Module', is directly supported by the provided text: '\_\_\_\_\_\_ == "\_\_main\_\_" Used to separate module usage and script execution. # file: greet.py def hello(): print("Hello!") if \_\_\_\_ == "\_\_main\_\_": hello() When greet.py is imported in another file, hello() won't run unless called directly. ■ 9.'

**Question No: 33 (Marks: 1)**

Which term best completes the following statement: 'Importing Modules Syntax: import module\_name Example: import math print(math.sqrt(16)) # Output: 4.0 Import \_\_\_\_\_ Items: from math import sqrt

`print(sqrt(25))` Import with Alias: `import math as m` `print(m.pi)` ■■ 3.?

**A. Specific**

- B. reused
- C. for
- D. your

**Explanation:**

The correct answer, 'Specific', is directly supported by the provided text: 'Importing Modules Syntax: `import module_name` Example: `import math` `print(math.sqrt(16))` # Output: 4.0 Import Specific Items: `from math import sqrt` `print(sqrt(25))` Import with Alias: `import math as m` `print(m.pi)` ■■ 3.'

### Question No: 34 (Marks: 1)

Which term best completes the following statement: 'Common Built-in Modules Module Description  
math Mathematical operations random \_\_\_\_\_ random numbers datetime Date and time manipulation  
Notes for CS306 introduction to python Noor institute of technology Module Description os Interacting  
with operating system sys System-specific parameters ■ 4.'?

A. square

**B. Generate**

- C. down
- D. output

**Explanation:**

The correct answer, 'Generate', is directly supported by the provided text: 'Common Built-in Modules Module Description math Mathematical operations random Generate random numbers datetime Date and time manipulation Notes for CS306 introduction to python Noor institute of technology Module Description os Interacting with operating system sys System-specific parameters ■ 4.'

### Question No: 35 (Marks: 1)

Which term best completes the following statement: 'The math Module Common Functions: Function  
`math.sqrt(x)` Description Square root `math.pow(x, y)` x raised to \_\_\_\_\_ power y `math.floor(x)` Round  
down `math.ceil(x)` Round up `math.pi` Value of  $\pi$  Example: `import math` `print(math.floor(4.7))` # Output: 4  
■■ 5.'?

A. time

B. explore

**C. The**

D. power

**Explanation:**

The correct answer, 'The', is directly supported by the provided text: 'The math Module Common Functions: Function `math.sqrt(x)` Description Square root `math.pow(x, y)` x raised to the power y `math.floor(x)` Round down `math.ceil(x)` Round up `math.pi` Value of  $\pi$  Example: `import math` `print(math.floor(4.7))` # Output: 4 ■■ 5.'

**Question No: 36 (Marks: 1)**

Which term best completes the following statement: '\_\_\_\_\_ is a Module? A module is a file containing Python code (functions, classes, variables) that can be reused in other programs. Any .py file is a module. ■ 2.'?

- A. create
- B. organize
- C. description

**D. What**

**Explanation:**

The correct answer, 'What', is directly supported by the provided text: 'What is a Module? A module is a file containing Python code (functions, classes, variables) that can be reused in other programs. Any .py file is a module. ■ 2.'.

**Question No: 37 (Marks: 1)**

Which term best completes the following statement: 'import, or aliases to access module content. Python includes many powerful built-in modules (math, random, etc.). You can create your own modules by saving Python code in .py files. \_\_name\_\_ == "\_\_main\_\_" ensures code only runs when executed directly. ■ \_\_\_\_\_ / Practice'?

- A. etc
- B. another

**C. Homework**

- D. content

**Explanation:**

The correct answer, 'Homework', is directly supported by the provided text: 'import, or aliases to access module content. Python includes many powerful built-in modules (math, random, etc.). You can create your own modules by saving Python code in .py files. \_\_name\_\_ == "\_\_main\_\_" ensures code only runs when executed directly. ■ Homework / Practice'.

**Question No: 38 (Marks: 1)**

Which term best completes the following statement: '■ Lecture Objectives ••••• Understand the concept of modules in Python. Learn how to import and use built-in and user-defined modules. Discover the use of the dir() \_\_\_\_\_. Understand the \_\_name\_\_ == "\_\_main\_\_" idiom. Explore the math and random modules. ■ 1.'?

- A. content

**B. Function**

- C. concept
- D. shuffle

**Explanation:**

The correct answer, 'Function', is directly supported by the provided text: '■ Lecture Objectives ••••• Understand the concept of modules in Python. Learn how to import and use built-in and user-defined modules. Discover the use of the dir() function. Understand the \_\_name\_\_ == "\_\_main\_\_" idiom. Explore the math and random modules. ■ 1.'

### Question No: 39 (Marks: 1)

Which term best completes the following statement: 'Polymorphism Polymorphism allows different classes to define methods with the same name, but possibly different behaviors. Example: class Bird: def fly(self): print("Bird can fly") class Airplane: def fly(self): print("Airplane flies in the sky") # Polymorphism using \_\_\_\_\_ def fly\_test(entity): entity.fly() b = Bird() a = Airplane() fly\_test(b) fly\_test(a) # Output: Bird can fly # Output: Airplane flies in the sky ■ 8.'?

A. multilevel  
B. world

**C. Function**

D. inherited

#### Explanation:

The correct answer, 'Function', is directly supported by the provided text: 'Polymorphism Polymorphism allows different classes to define methods with the same name, but possibly different behaviors. Example: class Bird: def fly(self): print("Bird can fly") class Airplane: def fly(self): print("Airplane flies in the sky") # Polymorphism using function def fly\_test(entity): entity.fly() b = Bird() a = Airplane() fly\_test(b) fly\_test(a) # Output: Bird can fly # Output: Airplane flies in the sky ■ 8.'

### Question No: 40 (Marks: 1)

Which term best completes the following statement: 'Dynamic Attributes Attributes can be added to an object at runtime. d.age = 5 print(d.age) # Output: 5 \_\_\_\_\_, this is not a common practice and may lead to inconsistent object structures. Notes for CS306 introduction to python Noor institute of technology ■ 8.'?

**A. However**

- B. title
- C. \_\_init\_\_
- D. each

#### Explanation:

The correct answer, 'However', is directly supported by the provided text: 'Dynamic Attributes Attributes can be added to an object at runtime. d.age = 5 print(d.age) # Output: 5 However, this is not a common practice and may lead to inconsistent object structures. Notes for CS306 introduction to python Noor institute of technology ■ 8.'

### Question No: 41 (Marks: 1)

Which term best completes the following statement: 'Working with Binary Files Binary \_\_\_\_\_ (e.g., images, audio files) require reading and writing in binary mode ("b"). Example: Reading a Binary File

with open("image.jpg", "rb") as file: content = file.read() print(content) Example: Writing to a Binary File with open("output.jpg", "wb") as file: file.write(content) Notes for CS306 introduction to python Noor institute of technology ■ Summary •••• Use open() to work with files in Python. Files can be opened in different modes ("r", "w", "a", etc.). Always close files after operations, or use the with statement to ensure automatic closure. Handle file exceptions using try and except blocks. ■ Homework / Practice 1. 2. 3.'?

A. time

B. several

### **C. Files**

D. root

#### **Explanation:**

The correct answer, 'Files', is directly supported by the provided text: 'Working with Binary Files Binary files (e.g., images, audio files) require reading and writing in binary mode ("b"). Example: Reading a Binary File with open("image.jpg", "rb") as file: content = file.read() print(content) Example: Writing to a Binary File with open("output.jpg", "wb") as file: file.write(content) Notes for CS306 introduction to python Noor institute of technology ■ Summary •••• Use open() to work with files in Python. Files can be opened in different modes ("r", "w", "a", etc.). Always close files after operations, or use the with statement to ensure automatic closure. Handle file exceptions using try and except blocks. ■ Homework / Practice 1. 2. 3.'.

#### **Question No: 42 (Marks: 1)**

Which term best completes the following statement: 'When you create an \_\_\_\_\_, Python calls the class constructor (\_\_init\_\_ method) to initialize the object's attributes. Creating an Object: person1 = Person("Alice", 30) person2 = Person("Bob", 25) Now, you can access the attributes and methods of the objects: print(person1.name) # Output: Alice person2.greet() # Output: Hello, my name is Bob and I am 25 years old. ■■ 3.'?

### **A. Object**

B. variables

C. inheritance

D. class

#### **Explanation:**

The correct answer, 'Object', is directly supported by the provided text: 'When you create an object, Python calls the class constructor (\_\_init\_\_ method) to initialize the object's attributes. Creating an Object: person1 = Person("Alice", 30) person2 = Person("Bob", 25) Now, you can access the attributes and methods of the objects: print(person1.name) # Output: Alice person2.greet() # Output: Hello, my name is Bob and I am 25 years old. ■■ 3.'.

#### **Question No: 43 (Marks: 1)**

Which term best completes the following statement: 'When Python encounters an error, it raises an exception to indicate that something went wrong. Types of \_\_\_\_\_ in Python:'?

A. cs306

B. indicate



### **C. Errors**

D. making

#### **Explanation:**

The correct answer, 'Errors', is directly supported by the provided text: 'When Python encounters an error, it raises an exception to indicate that something went wrong. Types of Errors in Python:'.

### **Question No: 44 (Marks: 1)**

Which term best completes the following statement: '■ Lecture \_\_\_\_\_ • • • • Understand what libraries are in Python. Learn how to import and use external libraries in Python. Explore some commonly used standard libraries in Python. Learn about third-party libraries and how to install them using pip. ■ 1.'?

- A. are
- B. install
- C. learn

### **D. Objectives**

#### **Explanation:**

The correct answer, 'Objectives', is directly supported by the provided text: '■ Lecture Objectives • • • • Understand what libraries are in Python. Learn how to import and use external libraries in Python. Explore some commonly used standard libraries in Python. Learn about third-party libraries and how to install them using pip. ■ 1.'.

### **Question No: 45 (Marks: 1)**

Which term best completes the following statement: 'It is used to initialize attributes. \_\_\_\_\_ : class Student: def \_\_init\_\_(self, name, roll\_no): self.name = name self.roll\_no = roll\_no Creating Objects: s1 = Student("Ali", 123) s2 = Student("Sara", 456) Accessing Attributes: print(s1.name) print(s2.roll\_no) # Output: Ali # Output: 456 ■ 4.'?

- A. adding

### **B. Example**

- C. object
- D. print

#### **Explanation:**

The correct answer, 'Example', is directly supported by the provided text: 'It is used to initialize attributes. Example: class Student: def \_\_init\_\_(self, name, roll\_no): self.name = name self.roll\_no = roll\_no Creating Objects: s1 = Student("Ali", 123) s2 = Student("Sara", 456) Accessing Attributes: print(s1.name) print(s2.roll\_no) # Output: Ali # Output: 456 ■ 4.'.

### **Question No: 46 (Marks: 1)**

Which term best completes the following statement: '■ Lecture \_\_\_\_\_ • • • • Understand the concept of modules in Python. Learn how to import and use built-in and user-defined modules. Discover the use

of the dir() function. Understand the `__name__ == "__main__"` idiom. Explore the math and random modules. ■ 1.?

A. noor

**B. Objectives**

C. common

D. only

**Explanation:**

The correct answer, 'Objectives', is directly supported by the provided text: '■ Lecture Objectives • • • • • Understand the concept of modules in Python. Learn how to import and use built-in and user-defined modules. Discover the use of the dir() function. Understand the `__name__ == "__main__"` idiom. Explore the math and random modules. ■ 1.'

**Question No: 47 (Marks: 1)**

Which term best completes the following statement: 'Protected Members Convention: prefix with a single underscore `_`. class Person: def `__init__`(self, name): self.\_name = name # protected class Student(Person): def display(self): Notes for CS306 introduction to python Noor institute of technology print("Name:", self.\_name) s = Student("Sara") s.display() # \_\_\_\_\_: Name: Sara Note: It's still accessible, but considered for internal use or subclassing. ■ 6.?'

**A. Output**

B. get\_gpa

C. controls

D. underscore

**Explanation:**

The correct answer, 'Output', is directly supported by the provided text: 'Protected Members Convention: prefix with a single underscore `_`. class Person: def `__init__`(self, name): self.\_name = name # protected class Student(Person): def display(self): Notes for CS306 introduction to python Noor institute of technology print("Name:", self.\_name) s = Student("Sara") s.display() # Output: Name: Sara Note: It's still accessible, but considered for internal use or subclassing. ■ 6.'

**Question No: 48 (Marks: 1)**

Which term best completes the following statement: '■ Lecture Objectives • Understand the concept of file handling in Python. Notes for CS306 introduction to python Noor institute of technology • • • \_\_\_\_\_ how to open, read, write, and close files in Python. Explore various modes for file operations in Python. Learn about context managers for safe file handling. ■ 1.?'

**A. Learn**

B. opened

C. cs306

D. object

**Explanation:**

The correct answer, 'Learn', is directly supported by the provided text: '■ Lecture Objectives • Understand the concept of file handling in Python. Notes for CS306 introduction to python Noor institute of technology • • • Learn how to open, read, write, and close files in Python. Explore various modes for file operations in Python. Learn about context managers for safe file handling. ■ 1.'

#### Question No: 49 (Marks: 1)

Which term best completes the following statement: 'Getter and \_\_\_\_\_ Methods Used to safely access or modify private attributes. Example: class BankAccount: def \_\_init\_\_(self): self.\_\_balance = 0 def get\_balance(self): return self.\_\_balance def set\_balance(self, amount): if amount >= 0: self.\_\_balance = amount else: print("Invalid amount") acc = BankAccount() acc.set\_balance(1000) print(acc.get\_balance()) # Output: 1000 ■ 8.'

**A. Setter**

- B. person
- C. decorator
- D. error

#### Explanation:

The correct answer, 'Setter', is directly supported by the provided text: 'Getter and Setter Methods Used to safely access or modify private attributes. Example: class BankAccount: def \_\_init\_\_(self): self.\_\_balance = 0 def get\_balance(self): return self.\_\_balance def set\_balance(self, amount): if amount >= 0: self.\_\_balance = amount else: print("Invalid amount") acc = BankAccount() acc.set\_balance(1000) print(acc.get\_balance()) # Output: 1000 ■ 8.'

#### Question No: 50 (Marks: 1)

Which term best completes the following statement: 'Introduction to Exceptions An \_\_\_\_\_ is an event that disrupts the normal flow of a program.'

A. importance

**B. Exception**

- C. python
- D. objectives

#### Explanation:

The correct answer, 'Exception', is directly supported by the provided text: 'Introduction to Exceptions An exception is an event that disrupts the normal flow of a program.'